# Numpy , Matpltlib, Scipy et les fichiers sous Python

Dr. Atef MASMOUDI (IPEIS AU 2015-2016)

## Manipulation des fichiers sous Python

Pour utiliser un fichier identifié par le chemin **<ch>** dans un programme Python, il faut commencer par l'ouvrir par l'appel de fonction
**open(<ch>, [<mode>])**
qui retourne un objet de type file.
Le paramètre facultatif **<mode>** indique le mode d'ouverture du fichier :
- **r** : mode lecture (le fichier doit exister préalablement)
- **w** : mode écriture (si le fichier existe, les données sont écrasées, sinon le fichier est créé)
- **a** : mode ajout (si le fichier existe, les données écrites vont l'être après celles existantes, sinon le fichier est créé)

Si le mode est omis, le mode par défaut est **r**.
Un objet de type file est associé à des attributs et des méthodes. En voici quelques-unes :
-**read([<n>])** : retourne la chaîne des <n> caractères restants
-**readline()** : lit une seule ligne à partir du fichier
-**readlines()** : utilise f.readline() de façon répétitive, et retourne une liste contenant toutes les lignes du fichier.
-**write(<ch>)** : écrit la chaîne de caractères <ch>
-**close()** : ferme le fichier
-**seek(<n>)** : choisit le caractère <n> comme position courante du fichier
-**tell()** : retourne le caractère en position courante

## Le module Scipy concerne le calcul scientifique

### Calcul d'intégrales

```
import numpy as np
import scipy.integrate as integ
def f(x):
    return x**2
lx=np.linspace(0,1,100)
ly=f(lx)
# méthode des trapèzes
s1=0
for I in range(10):
    s1+=(lx[i+1]-lx[i])*(ly[i]+ly[i+1])/2
# avec scipy.integrate
s2=integ.trapz(ly,lx) #attention à l'ordre (ly,lx)
#calcul de l'intégrale de f entre 0 et 1
s3=integ.quad(f,0,1)
```

### Calcul approché de f(x)=0

```
import scipy.optimize as resol
def f(x):
  return x**2-2
a=1
b=2
x=resol.bisect(f,a,b) # method de dichotomie
print("sol=",x)
x=resol.newton(f,a) # method de newton
print("sol=",x)
x=resol.fsolve(f,a)
print(" sol= ",x)
x=resol.root(f,a)
print(" sol= ",x)
```

### Résolution des Equations différentielles ordinaires

résolvons l'ODE dy/dt = -2y entre t=0..10, pour y(t=0)=1.

```
Import numpy as np
import scipy.integrate as integ
def f(y, t) : # une équation différentielle
    return -2*y
t = np.linspace(0, 10, 100)
y = odeint(f, 1, t) # solution
```

## Le module NumPy permet la manipulation simple et efficace des tableaux

```
import numpy as np
x = np.arange(a,b,PAS) # De a (inclus) à b (exclus) par pas de PAS
np.size(x) # Sa taille
np.shape(x) # dimension
x = np.array ([…]) # vecteur
x = np.array ([[..],[..],..,[..]]) # Matrice
x=np.zeros((m,n)) # matrice de 0 : m lignes et n colonnes
np.ones((m,n)) # matrice de 1 : m lignes et n colonnes
np.eye(n) # matrice identité de taille n*n
np.transpose(x) # transposé de x
x=np.linspace(a,b,n) #renvoie un array commençant par a, se terminant par b avec n éléments régulièrement espacés.
x=np.diag(v) # matrice diagonale dont la diagonale est le vecteur v
x=np.random.rand(n) ou bien ((n,p)) # vecteur (taille n), ou bien matrice (taille n,p) à coefficients aléatoires uniformes sur [0,1]
y = 2 * x # Multiplication de chaque terme
z = x + y # Sommation terme à terme
x=np.dot(a, b) # Produit de matrices
x[a:b] # de l'indice a à l'indice b exclu *
x[:b] # du début à l'indice b exclu
x[a:] # de l'indice a inclus à la fin
np.sum(x) # somme des éléments de x
np.prod(x) # produit des éléments de x
np.mean(x) # moyenne des éléments de x
np.min(x) #minimum de x
np.max(x) # maximum de x
np.pi # constante pi
np.linalg.det(x) # déterminant de x
np.linalg.inv(x) # matrice inverse de x
np.linalg.solve(A,b) # résoudre Ax=b
    D,V=np.linalg.eig(x) # D : valeurs propres, V : vecteurs propres
```

## Le module matplotlib sert à tracer et visualiser des données

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-np.pi,np.pi,100)
y = np.sin(x)
plt.plot(x, y,label= 'sin') # tracer une courbe qui relie des points dont les abscisses et ordonnées sont fournies dans les tableaux x et y respectivement.
plt.xlim(xmin, xmax) #fixer les domaines des abscisses
plt.ylim(ymin, ymax) #fixer les domaines des ordonnées
plt.title("sinus") # ajouter un titre
plt.legend() # ajouter une légende
plt. xlabel("x") # label de l'axe des abscisses
plt.ylabel("sin(x)") #label de l'axe des ordonnées
plt.axis("equal") #avoir la même échelle sur l'axe des abscisses et l'axe des ordonnées
plt.savefig("image.pgm") # sauvegarder la figure dans une image
plt.show() # affiche la figure a l'ecran
```

plt.subplot(m,n,p) ou bien plt.subplot(mnp) # m : nombre de lignes, n : nombre de colonnes et p est le numéro de la figure. Matplotlib place les figures au fur et à mesure dans le sens des lignes.